

Case Study Evaluation : CA-II (5 marks for each Question) Answer any two .

Q1. Analyze the Netflix case study: Challenge: In 2008, a major database corruption halted DVD shipments for three days, highlighting the fragility of their monolithic architecture and the need for a more resilient and scalable system.

Brainstorm What specific challenges, highlighted by the 2008 database corruption, did Netflix face with its monolithic architecture? How did their adoption of DevOps principles, including the migration to a microservices architecture and the pioneering of "Chaos Engineering," provide a solution to these problems and impact their service's resilience and scalability?

Q2. Amazon Challenge: In the early 2000s, Amazon's rapid growth was hampered by a monolithic architecture that led to frequent outages and slow feature releases, directly impacting customer satisfaction and revenue

Brainstorm how Amazon's adoption of 'two-pizza teams' and a microservices architecture directly addressed the challenges of their monolithic system, such as frequent outages and slow feature releases. How might this cultural and architectural shift have fostered the 'culture of continuous innovation' that allowed them to deploy code, on average, every 11.7 seconds?

Q3. Capital One Challenge: As a highly regulated financial institution, Capital One struggled with long release cycles (6-9 months) and the need to innovate in an increasingly digital market while maintaining stringent security and compliance standards.

Brainstorm how Capital One's "DevSecOps" approach, particularly the "shift-left" mentality and automated security scanning, addressed the dual challenges of long release cycles and stringent regulatory compliance. How can embedding security into the development pipeline, rather than treating it as a final gate, actually accelerate innovation and reduce production incidents in a highly regulated industry like finance?

<u>Task</u>	<u>Description</u>	<u>Marks</u>
<u>Task 1 – Deployment Strategy</u>	<u>Select one deployment tool (GitHub Actions / Codefresh / Harness / Spinnaker / etc.) and prepare a working pipeline for a simple service (ANY PBL OR BTECH PROJECT). Submit workflow file & pipeline diagram.</u>	<u>2</u>
<u>Task 2 – Configuration Management & IaC</u>	<u>Use Ansible or Puppet to configure the runtime environment (install packages, create users, manage files). Upload your Playbook/Manifest & inventory.</u>	<u>2</u>

<u>Task 3 – Containerization & Orchestration</u>	<u>Dockerize your service, create Kubernetes manifests (Deployment, Service). Demonstrate rolling updates & rollback. Provide YAMLS & screenshots.</u>	<u>2</u>
<u>Task 4 – Monitoring & Logging</u>	<u>Install Prometheus + Grafana (or Nagios). Expose app metrics & create a basic dashboard (uptime, latency, error rate). Upload dashboard screenshots.</u>	<u>2</u>
<u>Task 5 – Reflection & Report</u>	<u>4–5 slides: Architecture, pipeline flow, challenges, lessons learned. Documentation (Architecture, pipeline flow, challenges, lessons learned)</u>	<u>2</u>
<u>Bonus</u>	<u>Participate in an external DevOps challenge (Kaggle/Devpost/Cloud hackathon). Attach proof of submission/leaderboard or GitHub PR badge.</u>	